

张睿渤答辩闭环：词法分析与定位支持

1. 先记住自己的定位

你这部分的核心不是“把整个编译器讲完”，而是守住三件事：

1. 词法分析把字符流切成什么 token。
2. 注释、空白和大小写不敏感是怎么处理的。
3. 行列号怎么传给后续阶段，最后支撑 `caret diagnostics`。

最短自我介绍可以直接说：

我主要参与的是词法分析与定位支持这一层。词法分析器负责把 Pascal-S 源码切分成关键字、标识符、字面量、运算符等 token，同时跳过空白和注释，并记录每个 token 的行号和列号，供语法分析和错误诊断继续使用。

2. 现场先打开哪些文件

建议只记这 1 个文件，够用了：

1. [src/lexer.l](#)

如果老师追问接口关系，再补一句：

- 词法分析输出会通过 `yyllex()` 提供给 `yyparse()`。

打开顺序建议：

1. 先看 `YY_USER_ACTION`，讲“位置记录”。
2. 再看注释与空白规则，讲“跳过什么”。
3. 再看关键字、字面量、标识符规则，讲“翻译表”。
4. 最后看非法字符处理，讲“词法错误怎么进错误模块”。

3. 老师最可能问什么

问题 A：词法分析是怎么实现的？

先打开：

- [src/lexer.l:1](#)
- [src/lexer.l:44](#)

回答模板：

词法分析用的是 Flex。

我们在 `lexer.l` 里定义字符模式和 token 的对应关系，Flex 会根据这些规则生成扫描器。

扫描器逐段读取源代码，把关键字、标识符、字面量、运算符和分隔符转换成 token，再通过 `yyllex()` 提供给语法分析器。

问题 B：你们的“翻译表”可以怎么理解？

先打开：

- [src/lexer.l:55](#)
- [src/lexer.l:74](#)
- [src/lexer.l:92](#)
- [src/lexer.l:124](#)

回答模板：

翻译表可以理解成“什么字符模式对应什么 token”。

比如 `program`、`begin`、`end` 这些模式对应关键字 token；

`123`、`3.14`、`'a'` 对应整型、实型、字符字面量；

`:=`、`+`、`=` 等对应运算符；

逗号、分号、括号这些则对应分隔符 token。

问题 C：Pascal-S 不区分大小写，你们怎么处理？

先打开：

- [src/lexer.l:19](#)
- [src/lexer.l:136](#)

回答模板：

Pascal-S 是大小写不敏感的。

关键字规则直接用大小写混合字符类匹配，比如 `[Pp][Rr][Oo][Gg][Rr][Aa][Mm]`。

普通标识符则通过 `strdup_lower()` 统一转成小写再存入 `yyval`，这样后续语法和语义阶段都按统一名字处理。

问题 D：注释怎么处理？

先打开：

- [src/lexer.l:36](#)
- [src/lexer.l:45](#)

回答模板：

我们支持三类会被直接跳过的内容：空白、花括号注释 `{...}`、以及 `(* ... *)` 这种多行注释状态。

其中 `(* ... *)` 是通过独立的词法状态 `PAREN_COMMENT` 处理的，进入这个状态后不产生 token，只负责继续读到注释结束符并维护行列号。

另外还支持 `//` 行注释。

这题的关键是说清：

注释不会进入 parser，不会变成 token。

问题 E：与语法分析器的接口是什么？

先打开：

- [src/lexer.l:136](#)

回答模板：

接口就是 `yylex()`。

词法分析器每匹配到一个 token，就把 token 类型作为返回值交给 Bison，同时通过 `yyval` 传递对应语义值，比如名字、数字值、字符值等。

所以 parser 看到的不再是原始字符，而是 token 流加上必要的值。

问题 F：行列号怎么记录？为什么对错误诊断有帮助？

先打开：

- [src/lexer.l:13](#)
- [src/lexer.l:26](#)

回答模板：

`YY_USER_ACTION` 会在每次匹配 token 时先记录起始列号，再把当前列推进到 token 末尾。

这样后续 parser 和错误处理模块拿到 token 后，就知道它在源码中的具体位置。

对多行注释这类跨行 token，还会通过 `adjust_column_after_multiline_token()` 把列号调整到换行后的真实位置。

最后这些位置信息会被错误处理模块拿去输出 `caret diagnostics`。

问题 G：词法错误怎么处理？

先打开：

- [src/lexer.l:141](#)

回答模板：

如果字符既不属于已定义 token，也不属于空白或注释，就会走最后的兜底规则，直接调用

`ErrorHandler::lexical_error()`。

也就是说词法错误在这一层就会被识别出来，并带上行号、列号和长度信息。

4. 最该会指给老师看的 5 段代码

4.1 `YY_USER_ACTION`

位置：

- [src/lexer.l:13](#)

怎么讲：

这段是列号维护核心。每个 token 被识别出来时，先记下起始列，再把当前位置推进到 token 末尾。

4.2 PAREN_COMMENT 状态

位置:

- [src/lexer.l:36](#)
- [src/lexer.l:48](#)

怎么讲:

这部分专门处理 `( * ... * )` 注释。进入注释状态后不再产生 token，直到遇到结束符再回到正常扫描状态。

4.3 关键字规则

位置:

- [src/lexer.l:55](#)

怎么讲:

这里展示了大小写不敏感的关键字匹配规则，说明词法阶段已经把源语言关键字区分出来了。

4.4 标识符规则

位置:

- [src/lexer.l:136](#)

怎么讲:

这条规则除了返回 `IDENTIFIER`，还会把词素统一转成小写，并通过 `yyval` 交给 parser。

4.5 词法错误兜底

位置:

- [src/lexer.l:141](#)

怎么讲:

最后一条兜底规则用于识别非法字符，说明不是任何东西都会被默默吞掉。

5. 如果老师一个点一个点追问，怎么答

5.1 true/false 为什么不是普通关键字？

位置:

- [src/lexer.l:116](#)

回答:

我们把 `true/false` 直接识别成布尔字面量 token，并把值写进 `yyval.ival`。这样后续表达式和语义检查可以直接把它们当常量处理。

5.2 `read/readln` 为什么当成 IDENTIFIER？

位置：

- [src/lexer.l:119](#)

回答：

这几个名字在项目里按照内建过程来处理，所以词法层把它们当成特殊标识符交给后续阶段，而不是单独再开一整套语法类别。

5.3 词法阶段输出的到底是什么？

回答：

输出不是字符串，而是 token 类型加上语义值和位置信息。parser 直接消费这些 token。

6. 老师如果让你“讲设计思路”，你就这么收

我们这一层的设计思路是，先在词法阶段把 Pascal-S 源码切成稳定的 token 流，同时把大小写不敏感、注释跳过和位置信息记录都在这一层处理掉。这样 parser 看到的是更干净的输入，后面的语法分析和错误定位也更容易实现。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我主要参与的是词法分析与定位支持。我们用 Flex 按规则把 Pascal-S 源码切成关键字、标识符、字面量、运算符等 token，同时跳过空白和注释，并记录 token 的行号和列号。标识符会统一转成小写，保证 Pascal 的大小写不敏感规则。相关代码主要在 `lexer.l`。

8. 最容易说错的 4 个点

1. 不要把词法分析说成构造 AST。
正确说法：词法分析只输出 token。
2. 不要把注释说成会被 parser 看到。
正确说法：注释在 lexer 就被跳过了。
3. 不要把 `yyval` 说成只存字符串。
正确说法：它根据 token 类型存名字、数字、字符等不同语义值。
4. 不要忘了强调位置记录。
这是你这块和后续 `caret diagnostics` 最直接的连接点。

9. 张睿渤只需要记住的文件定位

最推荐只记下面这些：

- [src/lexer.l:13](#)
- [src/lexer.l:36](#)
- [src/lexer.l:55](#)
- [src/lexer.l:116](#)
- [src/lexer.l:136](#)
- [src/lexer.l:141](#)

这 6 处足够覆盖他的大部分答辩问题。